

M03 / PROTECTED ANSWERS

M03 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	4
How to use the protected answers	4
A01 - DE03-01 INNER JOIN and LEFT JOIN	4
Expected answer / validation	4
A02 - DE03-02 Cardinality, row multiplication and join validation	4
Expected answer / validation	4
A03 - DE03-03 UNION and UNION ALL	5
Expected answer / validation	5
A04 - DE03-04 String functions for cleaning and standardization	5
Expected answer / validation	5
A05 - DE03-05 Dates, timestamps and time analysis	5
Expected answer / validation	5
A06 - DE03-06 Subqueries and nested logic	6
Expected answer / validation	6
A07 - DE03-07 CTEs and readable multi-step SQL	6
Expected answer / validation	6
A08 - DE03-08 Window functions and PARTITION BY	6
Expected answer / validation	6
A09 - DE03-09 ROW_NUMBER, ranking and LAG/LEAD	7
Expected answer / validation	7
A10 - DE03-10 Advanced aggregation and multi-grain reasoning	7

Expected answer / validation	7
A11 - DE03-11 PostgreSQL setup, schemas and dialect awareness	7
Expected answer / validation	7
A12 - DE03-12 Constraints and transactions	8
Expected answer / validation	8
A13 - DE03-13 Indexes and EXPLAIN basics	8
Expected answer / validation	8
A14 - DE03-14 Views and reusable database logic	8
Expected answer / validation	8
A15 - DE03-15 Integrated unseen SQL challenge	9
Expected answer / validation	9
A16 - DE03-16 Timed SQL interview set and explanation	9
Expected answer / validation	9

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

PROTECTED TEST ANSWERS

Use this file to validate and repair, not to erase your first attempt. Compare the expected control with what you actually observed, close this guide, then solve a changed retry if the competency was weak.

Coverage: 16 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

How to use the protected answers

Open only the lesson you attempted. The guide gives the expected control/behavior, not permission to copy. If you used it, close it and perform a changed retry before marking the lesson mastered.

A01 - DE03-01 | INNER JOIN and LEFT JOIN

Expected answer / validation

```
SELECT o.order_id, c.customer_name
FROM course.orders AS o
LEFT JOIN course.customers AS c ON o.customer_id = c.customer_id
AND c.region = 'North'
ORDER BY o.order_id;
```

Six rows remain; only O105 has Iqbal's name. A WHERE restriction would keep just O105 because it removes null-extended and non-North matches after joining. The placement of a right-side rule can change which left records survive.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A02 - DE03-02 | Cardinality, row multiplication and join validation

Expected answer / validation

```
SELECT COUNT(*) AS result_rows, COUNT(i.line_no) AS matched_item_lines,
       COUNT(DISTINCT o.order_id) AS represented_orders
FROM course.orders AS o
LEFT JOIN course.order_items AS i ON o.order_id = i.order_id;
```

Results are 9, 8 and 6. O106 contributes a preserved result row with a null line_no, so COUNT(*) includes it while COUNT(i.line_no) does not. Three orders have multiple line rows, so there are more joined rows than distinct orders. Explain all three counts before accepting the join.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A03 - DE03-03 | UNION and UNION ALL

Expected answer / validation

```
SELECT category
FROM course.retry_sales
WHERE ordered_on = DATE '2026-11-01' INTERSECT
SELECT category
FROM course.retry_sales
WHERE ordered_on = DATE '2026-11-02';
```

```
SELECT category
FROM course.retry_sales
WHERE ordered_on = DATE '2026-11-01'
EXCEPT
SELECT category
FROM course.retry_sales
WHERE ordered_on = DATE '2026-11-02';
```

The intersection is Paper. The first-minus-second category is Writing. Reversing the EXCEPT branches changes the question and, here, produces no rows because November 2 contains only Paper, which also appears on November 1.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A04 - DE03-04 | String functions for cleaning and standardization

Expected answer / validation

```
SELECT ' Bag ' AS raw_value, NULLIF(LOWER(TRIM(' Bag ')),
                                     '') AS clean_value,
       LENGTH(NULLIF(LOWER(TRIM(' Bag ')), '')) AS clean_length
UNION ALL
SELECT 'BAG', NULLIF(LOWER(TRIM('BAG')), ''),
       LENGTH(NULLIF(LOWER(TRIM('BAG')), ''))
UNION ALL
SELECT ' ', NULLIF(LOWER(TRIM(' ')), ''),
       LENGTH(NULLIF(LOWER(TRIM(' ')), ''));
```

The first two cleaned values are bag with length 3. The third clean value and clean length are both NULL because the rule converts empty cleaned text to missing. If you wanted the length of the intermediate trimmed string, it would be zero; that is a different expression and a useful debugging distinction.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A05 - DE03-05 | Dates, timestamps and time analysis

Expected answer / validation

```
SELECT event_id
FROM course.events
WHERE happened_at >= TIMESTAMPTZ '2026-09-01 00:00:00+00'
      AND happened_at < TIMESTAMPTZ '2026-09-02 00:00:00+00'
ORDER BY happened_at;
```

T2 and T3 qualify. T1 is August 31 in UTC, while T3 is still September 1 in UTC. The source instants did not change; the definition of the reporting day changed. A date total without its timezone convention can be ambiguous.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A06 - DE03-06 | Subqueries and nested logic

Expected answer / validation

```
SELECT c.customer_id, c.customer_name
FROM course.customers AS c
WHERE
    (SELECT COUNT(*)
     FROM course.orders AS o
     WHERE o.customer_id = c.customer_id) >= 2
ORDER BY c.customer_id;
```

Only C01 Asha qualifies. EXISTS distinguishes zero from at least one; the condition here distinguishes fewer than two from at least two. The subquery must supply the kind of evidence the outer test needs.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A07 - DE03-07 | CTEs and readable multi-step SQL

Expected answer / validation

```
WITH item_totals AS
    (SELECT order_id, SUM(quantity * unit_price) AS item_gross
     FROM course.order_items
     GROUP BY order_id)
SELECT o.order_id
FROM course.orders AS o
LEFT JOIN item_totals AS i ON o.order_id = i.order_id
WHERE i.order_id IS NULL
ORDER BY o.order_id;
```

Only O106 appears. A missing right-side key proves no matching summary row exists here. An amount of zero could also come from actual zero-price lines, so the numeric amount alone is not a reliable existence test. Preserve the distinction between "no rows" and "rows summing to zero."

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A08 - DE03-08 | Window functions and PARTITION BY

Expected answer / validation

```
SELECT order_id,
    SUM(quantity * unit_price) OVER (
        ORDER BY ordered_on,
        order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_gross
FROM course.retry_sales
ORDER BY ordered_on, order_id;
```

R201 75, R202 95, R203 170 and R204 200. The first day's two records accumulate one at a time in ID order. The final running total reconciles to the ungrouped source total, while each intermediate number follows the specified ordering and frame.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A09 - DE03-09 | ROW_NUMBER, ranking and LAG/LEAD

Expected answer / validation

```
WITH previous AS
  (SELECT order_id, ordered_on, quantity * unit_price AS amount,
    LAG(quantity * unit_price) OVER (
      ORDER BY ordered_on,
      order_id) AS previous_amount
  FROM course.retry_sales)
SELECT order_id, amount, previous_amount,
  amount - previous_amount AS change_from_previous
FROM previous
ORDER BY ordered_on, order_id;
```

R201 is 75/NULL/NULL; R202 20/75/-55; R203 75/20/55; R204 30/75/-45. There is no earlier row for R201 in this source, so no measured change exists. Zero would mean "unchanged from a known prior value," which has not been established.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A10 - DE03-10 | Advanced aggregation and multi-grain reasoning

Expected answer / validation

```
WITH item_totals AS
  (SELECT order_id, SUM(quantity * unit_price) AS item_gross
  FROM course.order_items
  GROUP BY order_id)
SELECT COUNT(*) AS nonempty_orders, SUM(i.item_gross + o.shipping_fee) AS total_gross,
  ROUND(AVG(i.item_gross + o.shipping_fee), 2) AS avg_gross
FROM course.orders AS o
JOIN item_totals AS i ON o.order_id = i.order_id;
```

Five non-empty orders total 664, and 664 / 5 is 132.80. The inner join is deliberate because the question explicitly requires a matching item summary. It is not a general replacement for the left join in an all-order report. The change in denominator is the entire reason the average differs.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A11 - DE03-11 | PostgreSQL setup, schemas and dialect awareness

Expected answer / validation

```
CREATE TEMP TABLE desk_retry ( reference text PRIMARY KEY, units integer NOT NULL ); INSERT
INTO desk_retry (reference, units) VALUES ('X01', 3); SELECT reference, units FROM desk_retry;
```

Expect X01/3. Saving text alone does not create the object; execution does. The temporary table lasts for this session, whereas the saved .sql file remains wherever you saved it. If you need to repeat the same creation exercise, use a new session or a deliberate new temporary name.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A12 - DE03-12 | Constraints and transactions

Expected answer / validation

```
CREATE TEMP TABLE stock_retry ( product_id text PRIMARY KEY, units integer NOT NULL CHECK (units >= 0) ); INSERT INTO stock_retry VALUES ('ONE', 5); BEGIN; UPDATE stock_retry SET units = 4 WHERE product_id = 'ONE'; COMMIT; SELECT * FROM stock_retry;
```

The result is ONE/4. A second INSERT with product_id ONE fails because the primary key already exists. That failure does not remove or overwrite the original row. The committed update is outside a later transaction's rollback scope; reversing it would require another deliberate change, backed by the intended business rule.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A13 - DE03-13 | Indexes and EXPLAIN basics

Expected answer / validation

```
SELECT COUNT(*) AS matches  
FROM plan_orders  
WHERE customer_no = 21;
```

```
EXPLAIN  
SELECT order_no  
FROM plan_orders  
WHERE customer_no = 21;
```

There are 100 matches because remainders repeat once per hundred generated integers. The existing index organises order_no, not the customer_no expression. The planner commonly uses a sequential scan for this query in the fixture. The next engineering step would be to assess a real workload before proposing another index, not to index every column automatically.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A14 - DE03-14 | Views and reusable database logic

Expected answer / validation

```
BEGIN; UPDATE view_source SET units = 4 WHERE code = 'A'; SELECT * FROM view_demo;  
ROLLBACK; SELECT * FROM view_demo;
```

Inside the transaction the result is A/4/40. After rollback it returns to A/3/30. The view reads the source state visible to that session. It did not become a snapshot of 40 merely because you inspected it once.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A15 - DE03-15 | Integrated unseen SQL challenge

Expected answer / validation

```
WITH items AS
  (SELECT order_id, SUM(quantity * unit_price) AS amount
   FROM course.challenge_retry_items
   GROUP BY order_id),
  prepared AS
  (SELECT o.order_id, c.region,
         COALESCE(i.amount, 0) + o.shipping_fee AS order_gross
   FROM course.challenge_retry_orders AS o
   LEFT JOIN items AS i ON o.order_id = i.order_id
   LEFT JOIN course.challenge_retry_customers AS c ON o.customer_id = c.customer_id)
SELECT COALESCE(region, 'Unknown') AS region_group, COUNT(*) AS orders, SUM(order_gross) AS gross
FROM prepared
GROUP BY region
ORDER BY region_group;
```

R401 is $40 + 6 = 46$; R402 is $50 + 6 = 56$; R403 is empty with fee zero, total zero. East has two orders/102; Unknown has one/0. Three orders total 102, with items 90 and fees 12. The all-order mean is 34. Averaging East's 51 and Unknown's 0 equally would give 25.5 because it ignores their unequal counts. D11 and D12 have no orders.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.

A16 - DE03-16 | Timed SQL interview set and explanation

Expected answer / validation

```
WITH ranked AS
  (SELECT order_id, category, quantity * unit_price AS gross,
         ROW_NUMBER() OVER (PARTITION BY category
                           ORDER BY quantity * unit_price DESC, order_id) AS rn
   FROM course.practice_sales)
SELECT order_id, category, gross
FROM ranked
WHERE rn = 1
ORDER BY category;

SELECT category, SUM(quantity * unit_price) AS paid_gross
FROM course.practice_sales
WHERE is_paid = TRUE
GROUP BY category
HAVING SUM(quantity * unit_price) > 50
ORDER BY category;
```

The highest sales are Bags P104/120, Paper P102/80 and Writing P101/18. The paid-only threshold result is Paper/80; P104 is unknown-paid, so its larger amount is excluded by the stated rule. WHERE chooses input rows before grouping and window evaluation; filtering rn belongs in the outer query. The challenge_retry total is 102 across three orders, giving mean 34, with items 90 plus fees 12 as an independent check.

After repair: close this guide and repeat the competency on a changed input/scenario. Preserve the original failed/partial attempt.